

From Coder to Conductor

Running a fleet of coding agents without paying the **orchestration tax**: memory that outlives a session, handoffs that prove themselves, and decisions that wait for you.

DRAFT 0.1 · EDITION 2026.10 · CC BY 4.0

01

Summary

AI coding assistants are good at the next hour of work and bad at the next week. They forget what was decided, they stop when you close the laptop, and every tool you add is one more thing you carry in your head. The people who get furthest with agents stop treating them as a faster keyboard and start treating them as a team that needs structure: memory that outlives a session, handoffs that can be checked, messages that do not get lost, and a clear line between what an agent may do and what only you decide.

That structure is plumbing, and somebody has to build it. This paper describes the plumbing, part by part. We call the cost of building and running it yourself the *orchestration tax*. The paper also describes one open harness that already provides it: **OrchestraOS**, published by Tulum DAO under the Apache 2.0 license. It is honest about what that harness is today: a reference install that runs one operator's fleet every day, not a finished product.

Where to start:

- **If your assistant keeps forgetting what you decided,** read sections 2 and 3.
- **If you are about to write your own agent scripts,** read sections 4, 5 and 6.
- **If you are deciding whether this is sound enough to build on,** read sections 8 and 9.

02

Who this is for

This paper is written for three readers. You may be more than one of them.

You use an AI assistant every day, and it keeps forgetting. You explained the schema this morning, and by the afternoon it is inventing functions again. You have become the bridge between your editor, a chat tab and a terminal, and you cannot step away without the work stalling. Sections 2, 3 and 7 are for you.

You have been asked to make agents part of how your team works. You know what agent workflows are. What you do not want is a pile of fragile scripts that you now own forever, or agents that can touch things nobody approved. Sections 4, 5 and 6 are for you.

You are deciding whether to build on an open harness or write your own. You read the docs before the landing page, you check the issues and the license, and you are looking for the reason to say no. Sections 8 and 9 are for you.

A note on method: these reader profiles come from audience research that Tulum DAO commissioned. They describe situations, not people. Nothing in this paper is a customer quote.

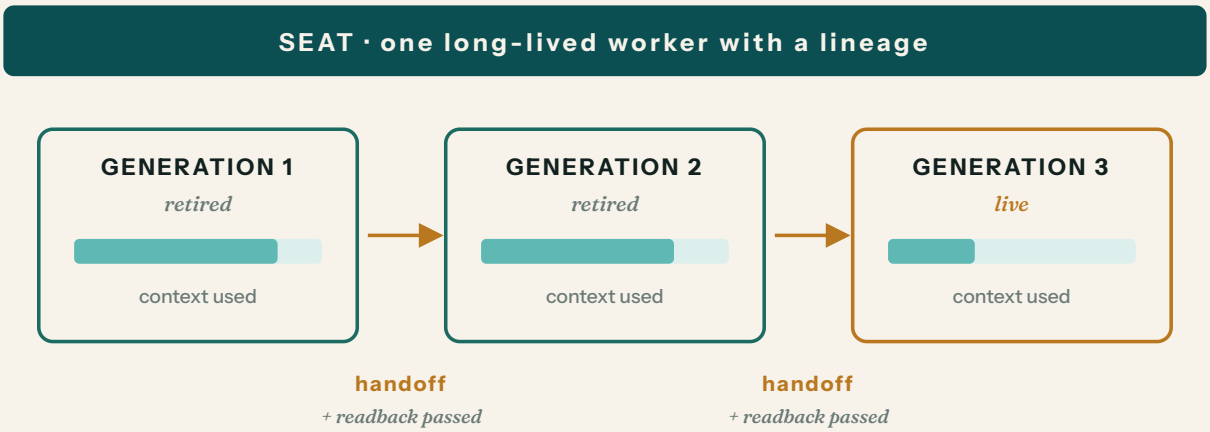
03

The context ceiling: why your assistant forgets

Every coding agent works inside a context window: the text it can hold at once. The window fills as the session goes on. Near the end, the agent either loses the oldest material or the session has to stop. What goes first is usually what mattered most: the constraint you set at the start, the reason behind the schema, the approach you already rejected. We call this the **Goldfish Effect**: the agent is still capable, but it no longer remembers why.

The common answer is to wait for a bigger window. That helps, and it is not enough. A window ten times larger still fills; it just fills later, and a long session that loses its early decisions fails in a way that is harder to notice. The problem is not only size. It is that a session is treated as the unit of work, when real work lasts longer than any session.

The fix is architectural. Treat each session as one **generation** of a longer-lived worker, called a **seat**. Before a generation runs out of room, it writes down what matters. A new generation starts fresh, reads that record, and carries on. The worker continues even though no single session does. OrchestraOS does this automatically: the rotation beat watches each armed seat's context and starts a handover before it is too late. It does this at a hard backstop near the top of the window, or earlier when the seat is idle *(S1, S2)*.



The work continues even though no single session does

A seat outlives its sessions. Each generation writes a handoff; the next one proves it understood before it takes over.

04

Handoffs that prove themselves

Writing a summary and starting a new session is not new. Anyone who has pasted "here is where we were" into a fresh chat has done it by hand. The difference between that and a dependable handover is **proof**.

In OrchestraOS, the generation that is about to retire writes a handoff (its *baton*). The baton holds the state of the work as checked by effect, the traps to avoid, the last commit, and a short set of questions. Each question's answer lives only in the retiring generation's own record of what happened. The successor must answer those questions, anchored to specific messages, turns or times, before it is promoted to be the live generation. A grader checks the answers. A successor that cannot show it understood does not take over *(S3, S4)*.

That gives you three properties a hand-pasted summary does not:

- **Lineage.** Every seat has a history of generations, with the runtime, model and session behind each one. You can always see which session did what *(S3)*.
- **One live generation.** Exactly one generation is the live one for a seat at any time. Everything that talks to the seat finds it through that pointer, so messages never go to a retired session *(S3)*.
- **Memory that survives rotation.** Each seat keeps a memory directory of small, single-fact files with an index, which its successor reads at boot. A shared facts store records each fact's source and freshness *(S5, S6)*.

Two limits matter here, and we state them plainly:

- **Runtimes.** Rotation is autonomous and on by default for **Claude Code** seats. For **Gemini** and **Codex** it is experimental and off unless you turn it on. Gemini has no reliable live reading of its context, so rotate those seats when they stop or by hand *(S2, S7)*.
- **Which seats.** The first wave arms worker seats, the T2 tier *(S8)*.

05

The orchestration tax

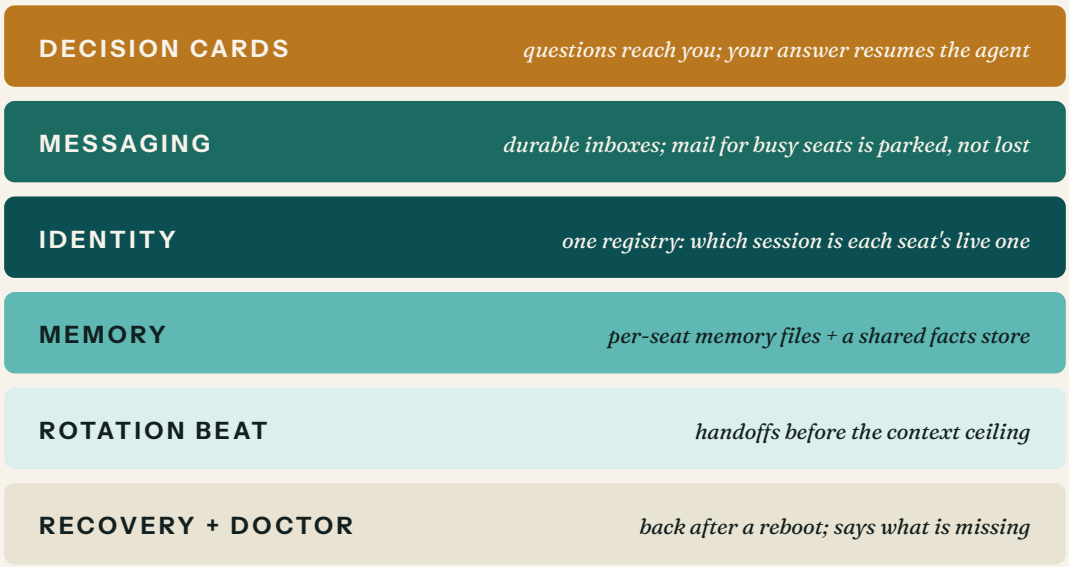
Suppose you decide to run several agents on real work. Within a few weeks, you will own the following, whether you planned to or not:

- **Messages that survive being busy.** Agents need to hand work to each other. If one is mid-task when a message arrives, the message must wait, not vanish. OrchestraOS has a durable message store with inboxes, acknowledgements and threads. A router delivers to idle seats and parks mail for busy ones, retrying rather than dropping it *(S9, S10)*.

- **One source of truth for who is who.** Sessions come and go. Something must know which session is a seat's live one, which runtime it uses, and how to resume it. Here that is a registry database. The flat file other tools read is generated from it, never edited by hand *(S3, S11)*.
- **A way for agents to ask you things.** Agents get stuck on decisions they should not make. That needs one channel for questions, answers that reach the agent that asked, and questions that never silently expire *(S12, S13)*.
- **Recovery after a reboot.** Servers restart. Your seats should come back from their saved sessions without you rebuilding them by hand *(S14)*.
- **Checks and upgrades.** A `doctor` that says what is missing, and an upgrade path that does not break running work *(S15)*.

That list is the **orchestration tax**: the time you spend building and maintaining the harness instead of the work the agents were meant to do. Every team that runs agents pays some of it. The question is whether you build the plumbing yourself or adopt an open version you can read, change and fix.

We make no claim here about how much time or money an open harness saves. We have no data for that, and you should distrust anyone who quotes a figure without it.



What every fleet ends up owning, built or adopted

The plumbing every fleet ends up owning. OrchestraOS ships each layer as an open, readable part.

06

Human-in-the-loop, on purpose

The fear behind most agent projects is a reasonable one: an agent doing something nobody approved. The answer is not to watch a terminal all day. It is to decide in advance which things only you decide, and

to make sure those questions reach you in a form you can answer quickly.

OrchestraOS treats every decision an agent needs from you as a **card**: approve or deny, pick from a menu, fill in a short questionnaire, or a "waiting on you" block when the next step is yours. Three rules apply:

- Decisions go on cards, never only in chat.
- Cards do not expire.
- Your answer resumes the agent that asked *(S12, S13)*.

The install guides mark the steps only a person may take, such as paying, signing in, typing any password and approving a device, as **[PERSON ONLY]**. They tell the agent helping you to stop there and let you act *(S16, S17)*.

Where you answer matters as much as how. **Today, cards render on a web dashboard that you open in any browser on your private network, including your phone's browser.** There are also iOS, watch and Mac apps, but they are **test builds and not released**, with no public download yet. Anything you read about approving from a phone app means the dashboard in your phone's browser until those apps ship *(S18, S19)*.

07

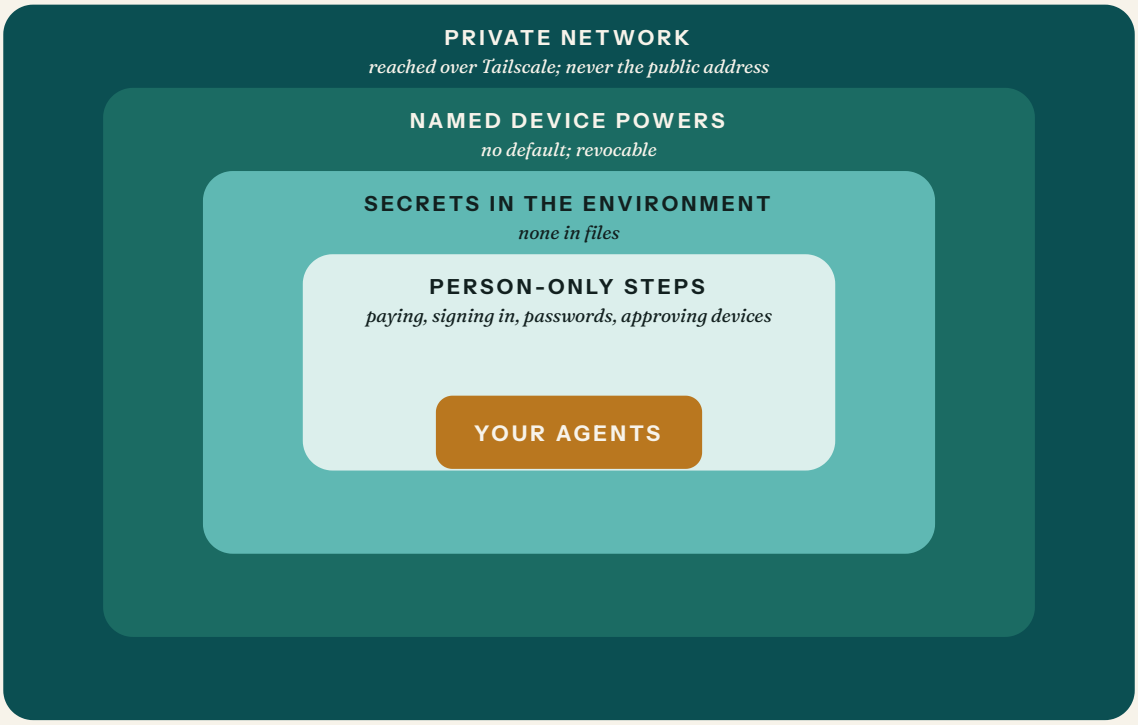
Blast radius: how the reference install fences agents

Before you let agents near your repositories, you will want to know how far a mistake can travel. These are the fences the reference install puts up. They are practices you can inspect in the code, not a certification or an audit.

- **Private by default.** You reach the dashboard and gateway over Tailscale, a private network between your own devices. The dashboard is never opened on the server's public address. Pairing refuses any address that has Tailscale Funnel turned on, because Funnel exposes it to the whole internet *(S20, S21)*.
- **Devices get only the powers you name.** Pairing a phone or computer requires you to list its powers: read, approve, message, and so on. There is no default, so nobody gets the power to answer on your behalf by accident. The docs say plainly that "approve" acts as you. Any device can be cut off with one command *(S22, S23)*.
- **Secrets stay out of files.** The configuration file holds no secrets. Bot tokens and API keys are read from the environment only *(S24)*.
- **The project checks itself.** Every pull request runs the tests and a secrets scan, and contributors sign off their commits *(S25)*.

- **Agents are told what never to do.** The install prompt tells any agent helping you never to delete, reset, overwrite or wipe anything, and to answer "no" to overwrite prompts *(S26)*.

None of this makes agents safe by itself. It makes the boundary visible, so you can decide where your own line is.



Fences, from the outside in: a private network, named device powers, secrets kept in the environment, and person-only steps.

08

From coder to conductor: a working day

This section is an illustration of how the parts fit together. It is not a customer story.

You start with a small team created by one command: a general manager seat, a project manager under it, and a worker under that *(S27)*. In the morning, you open the dashboard and tell Arturo, the assistant built into it, what you want done this week. Arturo writes the request into the general manager's inbox, and the work is broken down from there *(S28)*.

You do not watch the terminal. When the worker needs a decision, such as which of two approaches to take, or whether it may push, a card appears. You answer it from your laptop, or from your phone's browser over your private network, and the worker resumes. In the afternoon, the worker's session nears its context ceiling. Without you doing anything, a successor boots, answers questions about what

its predecessor was doing, and takes over. Tomorrow's session still knows why today's decisions were made.

Your job has changed. You write fewer lines and make more calls: what to build, what to approve, where the line is. That is the shift from coder to conductor. It does not remove the work of judgement. It moves the rest of the work out of your way.

If you want to know where your current setup stands before you change anything, the **Fleet audit prompt** in our first paper, *The Self-Tended Fleet* (tulumdao.com/whitepaper, section 2), is a copy-paste interview you can run with the AI you already use.

09

Vetting it: an honest spec sheet

If you are deciding whether to build on OrchestraOS, here is what it is today, with where each fact comes from.

QUESTION	ANSWER TODAY
What runs the agents?	One of three agent CLIs, installed and logged in on your server: **Claude Code**, **Codex**, or **Gemini** through Google's Antigravity `agy` CLI. A runtime catalog detects which ones you have and offers their models *(S29, S30)*
Does rotation work on all of them?	Supported on Claude Code. Experimental on Gemini and Codex, and off unless you turn it on *(S2, S7)*
Where does it run?	On your own server, which you rent; there is also a Docker path for trying it on a laptop. The models run on the vendors' servers, not yours *(S31, S32)*
What does it cost?	A small server and one agent plan. Prices change, so check the cost guide's sources. There is also a zero-key path for trying it: Gemini CLI's free tier on a VPS provider's new-account trial credit, $0 until the trial ends, then the server cost applies. Rotation on Gemini is experimental *(S32, S33, S7)*
License?	Apache 2.0 *(S34)*
How mature is it?	"Not a finished product." It runs one operator's fleet every day, and that setup is the reference install *(S35)*
Can I extend it?	Yes: chat bridges are plugins (Telegram today), there are thirteen open hackathon tracks, and contributions go through pull requests *(S36, S37)*

What it is not

- **Not a hosted service.** You run it on your own server.
- **Not production-certified.** It is a reference install and an open harness, changing in public.

- **Not for any model.** It drives three agent CLIs; it does not run local models.
- **Not a cost-cutting tool.** We have no data on savings and claim none.
- **Not on the App Store.** The phone and Mac apps are test builds.

10

Build or adopt: a checklist

Answer these about the setup you would build yourself. Each one maps to a section of this paper.

1. When a session runs out of context, what carries the work forward, and how do you know the next session understood it? *(section 3)*
2. If two agents message each other and one is busy, where does the message wait? *(section 4)*
3. Which session is the live one for each worker, and where is that written down? *(section 4)*
4. When an agent needs a decision, how does the question reach you away from your desk, and does your answer restart the agent? *(section 5)*
5. Which actions can only a person take, and is that list enforced or just remembered? *(section 5)*
6. What can a stolen phone do on your system? *(section 6)*
7. After a reboot, what brings your agents back? *(section 4)*
8. Who maintains all of the above when the vendors change their CLIs? *(sections 4 and 8)*

If you have good answers to all eight, you may not need an open harness. If you do not, each "no" is a part of the orchestration tax you will pay one way or another.

11

Start this week

One step for each reader:

- **If your assistant keeps forgetting:** give the install playbook to the AI you already use. It walks you from renting a server to your dashboard and stops only where you must act yourself: github.com/Tulum-DAO/orchestraos, file `docs/AGENT_INSTALL.md` *(S38)*.
- **If you are building agent workflows for a team:** run the seven-step gate in `docs/GATE.md`. It is the same first test everyone completes *(S39)*.
- **If you are vetting it:** read `docs/ARCHITECTURE.md`, then reproduce the rotation proof on a clean install, as described in `docs/ROTATION.md` *(S40)*.

Everything lives in one public repository: **github.com/Tulum-DAO/orchestraos**.

12

Source notes

Every claim above about OrchestraOS is drawn from the public repository at commit **`152c0a6`** (2026-10-14). Line numbers refer to that commit. This is a living document: when the repository changes, the notes are re-checked, and claims that no longer hold are corrected in the next edition.

- **S1** `docs/ROTATION.md:1-8`: rotation, seats, generations, autonomous by default.
- **S2** `docs/ROTATION.md:46-49`: swap at a hard backstop (0.80) or an idle lull (0.70); README.md:87.
- **S3** `docs/ARCHITECTURE.md:27-32`: seat, generation, canonical, baton, readback, card.
- **S4** `docs/ARCHITECTURE.md:68`: "lossless only if the successor answers the canary".
- **S5** `README.md:89`: memory.
- **S6** `docs/ARCHITECTURE.md:57-58`: per-seat memory directories and the facts store.
- **S7** `docs/ROTATION.md:131-142`: runtime support and known gaps.
- **S8** `docs/ROTATION.md:27`: wave-1 armed tier is T2.
- **S9** `README.md:88`: inter-agent messaging.
- **S10** `docs/ARCHITECTURE.md:36-40`: message store and router; parked rows are retried.
- **S11** `docs/ARCHITECTURE.md:42-43, :66`: spawn and registry; the DB is the identity truth.
- **S12** `README.md:90`: the approvals surface.
- **S13** `docs/ARCHITECTURE.md:32, :45-46, :67`: cards never expire; approvals; decisions go on cards.
- **S14** `README.md:76-83`: start on boot and recovery.
- **S15** `README.md:100`: upgrading and doctor.
- **S16** `README.md:33-35`: [PERSON ONLY] steps.
- **S17** `docs/INSTALL.md:8`: [PERSON ONLY] in the install guide.
- **S18** `README.md:66`: phone and Mac apps in testing, not yet public.
- **S19** `docs/ONBOARDING.md:43-44`: iOS and Mac apps not released yet.
- **S20** `docs/INSTALL.md:59-60`: the dashboard is never opened on the public IP.
- **S21** `docs/ONBOARDING.md:17, :384`: Funnel addresses never used; keep it tailnet-only.
- **S22** `docs/ONBOARDING.md:197-203`: device powers required, no default; approve acts as you.
- **S23** `docs/ONBOARDING.md:226`: revoking a device.
- **S24** `README.md:108`: secrets read from the environment only.
- **S25** `README.md:138`: CI tests, secrets scan, DCO sign-off.
- **S26** `README.md:36-37`: never delete, reset, overwrite or wipe.
- **S27** `README.md:53`: `orchestra starter` creates gm, a project manager and a worker.
- **S28** `docs/ARCHITECTURE.md:51-52`: Arturo commissions work through the general manager's inbox.
- **S29** `README.md:69-70`: the three agent CLIs.
- **S30** `README.md:92`: runtime catalog.
- **S31** `README.md:62-64`: install paths, including Docker.
- **S32** `docs/COSTS.md:1-14`: costs; the model runs on the vendor's servers.
- **S33** `docs/COSTS.md:114-121`: the zero-key path (Gemini free tier + VPS trial credit; budget the server after the trial).
- **S34** `README.md:142`: Apache 2.0.
- **S35** `README.md:56`: "This is not a finished product."
- **S36** `plugins/telegram`: a chat bridge as a plugin.
- **S37** `README.md:110-138`: hackathon tracks and how to contribute.
- **S38** `README.md:9-12`: the agent install playbook.
- **S39** `README.md:96-97`: the seven-step gate.
- **S40** `docs/ROTATION.md:165`: reproduce the Claude proof on a clean install.

Editions and changes

From Coder to Conductor is a living document. The argument stays fixed between editions; the facts about OrchestraOS are re-checked against the repository and corrected when they change. Each edition is a dated snapshot, and its PDF carries the edition in its file name.

EDITION	WHAT CHANGED
2026.10 (Draft 0.1)	First draft for review

Related reading

- *The Self-Tended Fleet*, Tulum DAO: an autonomy maturity model for operators, with the Fleet audit prompt. tulumdao.com/whitepaper.
- The OrchestraOS repository and docs: github.com/Tulum-DAO/orchestraos.

License

© 2026 Tulum DAO. *From Coder to Conductor* is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, including commercially, as long as you give appropriate credit to Tulum DAO, link to the license (creativecommons.org/licenses/by/4.0), and indicate if changes were made.